



Unit 5

AJAX & Extensible Markup Language

AJAX



- AJAX stands for Asynchronous JavaScript and XML, and it describes a set of development techniques used for building websites and web applications. With the help of AJAX user's web browser doesn't need to reload an entire webpage when only a small portion of content on the page needs to change. With the help of AJAX we can:
- Update a web page without reloading the page
- Request data from a server after the page has loaded
- Receive data from a server after the page has loaded
- Send data to a server in the background.



Working of AJAX

- An event occurs in a webpage(for e.g. a button is clicked)
- An XMLHttpRequest object is created by JavaScript
- The XMLHttpRequest object sends a request to web server.
- The server processes the request and sends response back to the web page
- The response is read by JavaScript and proper action is performed by JavaScript(like page update).



Advantage of AJAX

- Speed: It reduces server traffic in both side request as well as time consuming on both side response
- Interactive
- Asynchronous calls
- Form Validation
- Bandwidth Usage



Disadvantage of AJAX

- Search engines would not be able to index an AJAX application
- The server information can not be accessed within AJAX
- AJAX is not well integrated with any browser
- ActiveX requests are enabled only in IE5 and IE6.
- Data of all requests is URL encoded which increases size of the request.



XMLHttpRequest Object

- All the modern browsers support the XMLHttpRequest object. The XMLHttpRequest object can be used to exchange data with the server.
E.g. **var xhr=new XMLHttpRequest();** //declaring request object
- **AJAX Request:** To send a request to a server, we use the open() and send() methods of the XMLHttpRequest object.
- E.g. **xhr.open("GET","user_info.txt",true);**
// (method, URL, async)
xhr.send();
// send(string) for POST



XMLHttpRequest Object

- **AJAX Response:** The `responseText` property returns the server response as JavaScript string, and we can use it accordingly:
- Example:
`document.getElementById("demo").innerHTML=xhr.responseText;`
- The `responseXML` property returns the server response as an XML DOM object. `getResponseHeader()` and `getAllResponseHeader()` are the server response methods.



XML (Extensible Markup Language)

- XML (Extensible Markup Language) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. It was designed to store and transport data in a structured manner.
- XML stands for eXtensible Markup Language
- XML is a markup language much like HTML
- XML was designed to store and transport data not display
- XML was designed to be self-descriptive
- Platform independent
- XML is a W3C Recommendation



Applications/Use of XML

Vector Markup language(VML) : It is a language based on XML, which allows us to describe and format vectors. It was designed to help the developers address book problems with binary graphic files. The great application of VML is the creation of menus and navigational elements.

XML based variant configuration language(XVCL): Variants generally occurs in software reuse. XVCL allows us to configure product variants from common core generic, reusable meta components.



Applications/Use of XML

- SMIL 2.0: It allows us to use a text editor to create reusable content such as audio, video that can be delivered through web.
- XML based directories for MXF/AAF applications: Advanced Authoring Format(AAF) is a binary file format for post production environment. It has a model that can be used for describing video, audio etc.

Example



The line `<?xml version="1.0" encoding="UTF-8"?>` is called the XML declaration. It is used to specify the version of XML being used and the character encoding used in the XML document.

```
<note>  
  <to>Raj</to>  
  <from>Ravi</from>  
  <heading>Reminder</heading>  
  <body>Meeting at 8am.</body>  
</note>
```



Syntax rules for XML documents

- **Rule 1: All XML must have a Root Element.**
- A root element is simply a set of tag that contains our XML content. In example below <book> is root element.
- Example: <book>
 <author>Stephan Hawking</author>
 <price>1200</price>
 </book>



Syntax rules for XML documents

- **Rule 2: All tags must be closed:**
- If we open any tag in some order then we need to close those tags. In above example opening tags `<book>`, `<author>`, `<price>` are closed with closing tags `</book>`, `</author>`, `</price>`
- **Rule 3: All tags must be properly nested:**
- When we nest one tag within another, we should pay attention to the order in which we open each tag, and then we close tags in the reverse order.
- Example : `<book><author></author></book>`



Syntax rules for XML documents

- **Rule 4: Tag names are case sensitive:**
 - Uppercase and lowercase matter in XML Opening and closing tags must match exactly. Ex.<BOOK>,<Book>,<book> are all different tags
- **Rule 5: Tag Names cannot contain spaces**
- **Rule 6: Attribute values must appear within quotas:**
 - Example:<artist title="Leonardo">



Syntax rules for XML documents

- **Rule 7: White Space is Preserved.**
- In HTML tag, incase of more than one white space, extra white spaces are ignored but in XML the spaces are preserved.
- Ex. <book>The legend of Zorro</book>
- **Rule 8: Tag names restrictions:**
- Tag names can't start with the letters *xml* , a number , or punctuation except for the underscore (_).



Syntax rules for XML documents

- **Rule 9: Avoid HTML tags(Optional)**
- **Ex. <p><td>**
- **Nested Syntax of XML Element**

<root>

<child>

<subchild>.....</subchild>

</child>

</root>



XML Element and Attribute

- In XML, elements and attributes are two fundamental components used to structure and represent data. Here's a breakdown of elements and attributes in XML:

Elements:

- Elements are the building blocks of an XML document.
- They represent the different parts or components of the data being described.
- Elements are defined by tags enclosed in angle brackets (< and >).
- They have an opening tag, a closing tag, and content in between.
- The opening tag marks the beginning of the element, while the closing tag marks the end.
- Elements can have child elements, nested elements, and text content.
- Elements can also have attributes associated with them.
- Example: `<book>Harry Potter</book>`



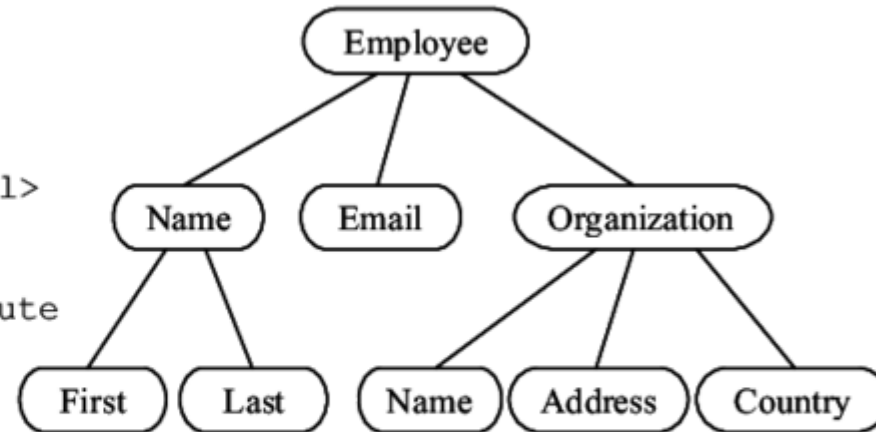
XML Attribute

- Attributes provide additional information or metadata about elements.
- They are defined within the opening tag of an element.
- Attributes are written as key-value pairs, separated by an equal sign (=).
- The attribute value is enclosed in quotes (" or ').
- Attributes provide characteristics, properties, or descriptive data related to the element.
- Attributes are used to enhance the understanding and processing of the element.
- Example: `<book id="123" language="en">`

XML Tree



```
<Employee>
  <Name>
    <First>Lassi</First>
    <Last>Lehto</Last>
  </Name>
  <Email>Lassi.Lehto@fgi.fi</Email>
  <Organization>
    <Name>
      Finnish Geodetic Institute
    </Name>
    <Address>
      PO Box 15,
      FIN-02431 Masala
    </Address>
    <Country CountryCode="358">Finland</Country>
  </Organization>
</Employee>
```



XML Tree

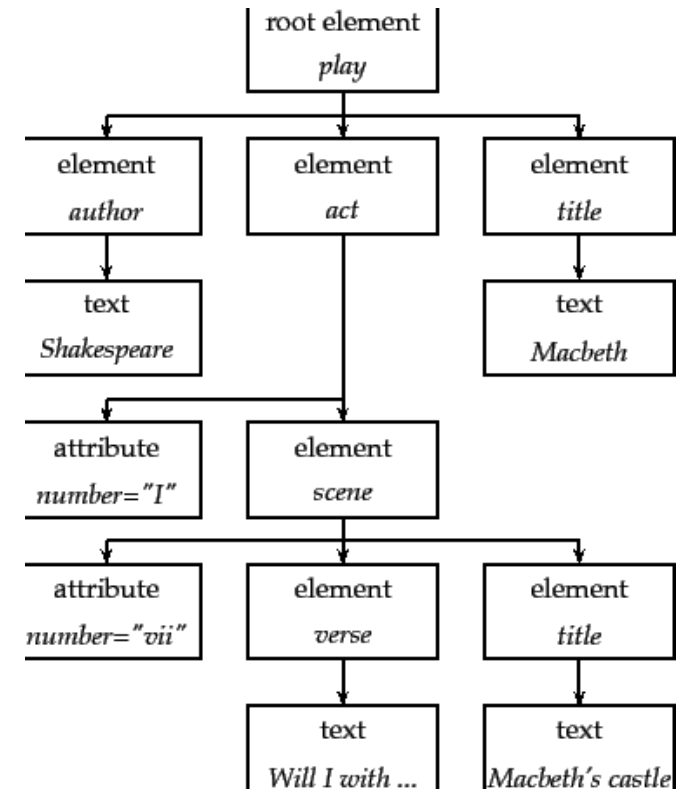


- XML document has a self descriptive structure. It forms a tree structure which is referred as an XML tree.
- Tree structure contains root element(as parent), child element and so on. It is very easy to traverse all succeeding branches and sub branches and leaf nodes starting from the root

Assignments



Write xml code for given xml tree





XML Namespace

- XML Namespace is a mechanism to avoid name conflicts by differentiating elements of attributes within an XML document that may have identical names but different definitions. XML data has to be exchanged between several application. Same tag name may have different meanings in different applications. So, it creates confusion on exchanging documents. So, XML namespace is used in XML file as:
Using a prefix & using xmlns attribute



XML Namespaces - The xmlns Attribute

- When using prefixes in XML, a namespace for the prefix must be defined.
- The namespace can be defined by an xmlns attribute in the start tag of an element.
- The namespace declaration has the following syntax.
`xmlns:prefix="URI".`



XML Namespaces-Example

- Consider two XML fragment:

- //1.xml

```
<table>
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

- //2.xml

```
<table>
  <width>80</width>
  <length>120</length>
</table>
```

If these XML fragments were added together, there would be a name conflict. Both contain a `<table>` element, but the elements have different content and meaning.



XML Namespaces-Example

Now XML namespace is used in XML file as: Example:

```
<root>

    <h:table xmlns:h="https://w3.org">
        <h:tr>
            <h:td>Apples</h:td>
            <h:td>Bananas</h:td>
        </h:tr>
    </h:table>
    <f:table xmlns:f="https://w3schools.com">
        <f:width>80</f:width>
        <f:length>120</f:length>
    </f:table>

</root>
```

In this example there will be no conflict because the two <table> elements have now different names.

Example



```
<root>

<h:table xmlns:h="http://www.w3.org/TR/html4/">
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table xmlns:f="https://www.w3schools.com/furniture">
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>

</root>
```

In the example above:

The xmlns attribute in the first <table> element gives the h: prefix a qualified namespace.

The xmlns attribute in the second <table> element gives the f: prefix a qualified namespace.

When a namespace is defined for an element, all child elements with the same prefix are associated with the same namespace.

Namespaces can also be declared in the XML root element:

Assignment



- Write use of xml namespace to merge following 2 xml files.

```
<detail>
  <name>Mango</name>
  <price>200</price>
</detail>
```

```
<detail>
  <name>Bir Bahadur</name>
  <address>Kalanki</address>
<detail>
```

DTD



- DTD stands for Document Type Definition.
- A DTD is XML schema language that defines the structure and the legal elements and attributes of an XML document.
- XML document with correct syntax is called "Well Formed".
- An XML document validated against a DTD is both "Well Formed" and "Valid".

Document Type Definition



- Basic syntax of DTD is as follow:

```
<?DOCTYPE element DTD  
identifier
```

```
[
```

```
    declaration1
```

```
    declaration2
```

```
    .....
```

```
]>
```

In given syntax:

- **DTD** starts with `<!DOCTYPE` delimiter
- An **element** tells the parser to parse document from specified root element.
- **DTD identifier** is an identifier for the document type definition, which may be the path to a file on the system or URL to a file on the internet. If the DTD is pointing to external path, it is called external subset.
- The **square brackets[]** enclose an optional list of entity declaration called internal subset

Internal DTD



- Syntax:
- <!DOCTYPE root-element[element-declarations]>
- DTD scripts are written inside same xml file.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

External DTD Example



In external DTD elements are declared outside the XML file. They are accessed by specifying the system attributes which may be either the DTD file or a valid URL. Syntax `<!DOCTYPE root-element SYSTEM "file-name">`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

Note.dtd:

```
<!DOCTYPE note
[
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
```



DTD example Explanation

- The DTD above is interpreted like this:
- !DOCTYPE note - Defines that the root element of the document is note
- !ELEMENT note - Defines that the note element must contain the elements: "to, from, heading, body"
- !ELEMENT to - Defines the to element to be of type "#PCDATA"
- !ELEMENT from - Defines the from element to be of type "#PCDATA"
- !ELEMENT heading - Defines the heading element to be of type "#PCDATA"
- !ELEMENT body - Defines the body element to be of type "#PCDATA"
- **Tip:** #PCDATA means parseable character data.



Assignments

- Mention differences between internal and external DTD??
- Write internal and external DTD code for following XML document.

```
<brand>  
    <model>BT200</model>  
    <price>2000</price>  
    <quantity>2</quantity>  
</brand>
```

DTD Elements



- Declaring elements
- `<!ELEMENT element-name category>`
- Or
- `<!ELEMENT element-name (element-content)>`



DTD Elements

Empty Elements

Empty elements are declared with the category keyword EMPTY:

```
<!ELEMENT element-name EMPTY>
```

Example:

```
<!ELEMENT br EMPTY>
```

XML example:

Elements with Parsed Character Data

Elements with only parsed character data are declared with #PCDATA inside parentheses:

```
<!ELEMENT element-name (#PCDATA)>
```

Example:

```
<!ELEMENT user (#PCDATA)>
```

//here user is tag name <user>someone's name </user>

DTD Elements



Elements with any Contents

Elements declared with the category keyword *ANY*, can contain any combination of parsable data:

```
<!ELEMENT element-name ANY>
```

Example:

```
<!ELEMENT note ANY>
```



DTD Elements

Elements with Children (sequences)

Elements with one or more children are declared with the name of the children elements inside parentheses:

```
<!ELEMENT element-name (child1)>
```

or

```
<!ELEMENT element-name (child1,child2,...)>
```

Example:

```
<!ELEMENT note (to,from,heading,body)>
```

When children are declared in a sequence separated by commas, the children must appear in the same sequence in the document. In a full declaration, the children must also be declared, and the children can also have children. The full declaration of the "note" element is:

```
<!ELEMENT note (to,from,heading,body)>
```

```
<!ELEMENT to (#PCDATA)>
```

```
<!ELEMENT from (#PCDATA)>
```

```
<!ELEMENT heading (#PCDATA)>
```

```
<!ELEMENT body (#PCDATA)>
```

DTD Element operators



... or operators/syntax rules we can use when defining child elements:

Operator	Syntax	Description
+	a^+	One or more occurrences of a
*	a^*	Zero or more occurrences of a
?	$a^?$	Either a or nothing
.	a, b	a followed by b
	$a \mid b$	a or b
()	(expression)	An expression surrounded by parentheses is treated as a unit and could have any one of the following suffixes?, *, or +.



DTD Elements Operators

Declaring Only One Occurrence of an Element

```
<!ELEMENT element-name (child-name)>
```

Example: `<!ELEMENT note (message)>`

The example above declares that the child element "message" must occur once, and only once inside the "note" element.

Declaring Minimum One Occurrence to two occurrence of an Element(+)

```
<!ELEMENT element-name (child-name+)>
```

Declaring Zero or More Occurrences of an Element(*)

```
<!ELEMENT element-name (child-name*)>
```

Declaring Zero or One Occurrences of an Element(?)

```
<!ELEMENT element-name (child-name?)>
```

Declaring either/or Content

```
<!ELEMENT note (to,from,header,(message|body))>
```

The example above declares that the "note" element must contain a "to" element, a "from" element, a "header" element, and either a "message" or a "body" element.

DTD attributes



- **<!ATTLIST element-name attribute-name attribute-type attribute-value>**
- DTD example:
- `<!ATTLIST payment type CDATA "check">` // default value of type will be check
- XML example:
- `<payment type="check">`

DTD attributes



The **attribute-type** can be one of the following:

Type	Description
CDATA	The value is character data
(<i>en1 en2 ..</i>)	The value must be one from an enumerated list
ID	The value is a unique id
IDREF	The value is the id of another element
IDREFS	The value is a list of other ids
NMTOKEN	The value is a valid XML name
NMTOKENS	The value is a list of valid XML names
ENTITY	The value is an entity
ENTITIES	The value is a list of entities
NOTATION	The value is a name of a notation
xml:	The value is a predefined xml value

The **attribute-value** can be one of the following:

Value	Explanation
<i>value</i>	The default value of the attribute
#REQUIRED	The attribute is required
#IMPLIED	The attribute is optional
#FIXED <i>value</i>	The attribute value is fixed

DTD attributes



- Value for number is required.

DTD:

```
<!ATTLIST person number CDATA #REQUIRED>
```

Valid XML:

```
<person number="5677" />
```

Invalid XML:

```
<person />
```

- Value of type can be check or cash only

DTD:

```
<!ATTLIST payment type (check|cash) "cash">
```

XML example:

```
<payment type="check" />
```

or

```
<payment type="cash" />
```

Assignment



- Write valid DTD xml for following:

```
<library>
  <sciencesection>
    <book>
      <name>Biology</name>
      <author>D.K.Khadka</author>
    </book>
    <book>
      <name>Physics</name>
      <author>R.R.Karmacharya</author>
    </book>
  </sciencesection>
  <accountsection>
    <book>
      <name>Manegerial Accounting</name>
      <author>G.P. Amatya</author>
    </book>
  </accountsection>
</library>
```

DTD assignment



- Write a valid xml for given DTD.

```
<!DOCTYPE TVSCHEDULE [  
  
  <!ELEMENT TVSCHEDULE (CHANNEL+)>  
  <!ELEMENT CHANNEL (BANNER, DAY+)>  
  <!ELEMENT BANNER (#PCDATA)>  
  <!ELEMENT DAY (DATE, (HOLIDAY|PROGRAMSLOT+)+)>  
  <!ELEMENT HOLIDAY (#PCDATA)>  
  <!ELEMENT DATE (#PCDATA)>  
  <!ELEMENT PROGRAMSLOT (TIME, TITLE, DESCRIPTION?)>  
  <!ELEMENT TIME (#PCDATA)>  
  <!ELEMENT TITLE (#PCDATA)>  
  <!ELEMENT DESCRIPTION (#PCDATA)>  
  
  <!ATTLIST TVSCHEDULE NAME CDATA #REQUIRED>  
  <!ATTLIST CHANNEL CHAN CDATA #REQUIRED>  
  <!ATTLIST PROGRAMSLOT VTR CDATA #IMPLIED>  
  <!ATTLIST TITLE RATING CDATA #IMPLIED>  
  <!ATTLIST TITLE LANGUAGE CDATA #IMPLIED>  
]>
```

Assignment



- Write internal and external DTD to describe "mail" as a root element and "to", "from", and "subject", and "message" as child elements.
- Write DTD script demonstrating use of different DTD operators.
- Write DTD script demonstrating use of different DTD attribute values.

XML Schema or XML Schema Definition (XSD)



- In XML, a schema is a structural definition that specifies the rules, constraints, and data types for an XML document. XML schemas define the structure and content of XML documents, allowing validation, data integrity, and interoperability.
- It is like DTD but provides more control on XML structure



XSD Definition Types

XML Definition types:

Simple Type: Used only in the context of the text. Some of predefined simple types are: `xs:integer`, `xs:Boolean`, `xs:string`, `xs:date`

Ex. `<xs:element name="student_name" type="xs:int" />`

Complex Type: It is a container for other element definitions. This allows us to specify which child elements an element can contain and to provide some structure within our XML documents. In the below example, `contact` element consists of child elements. This is a container for other `<xs:element>` definitions, that allows us to build a simple hierarchy of elements in the XML document.



XSD Example

Add.xsd

```
<?xml version="1.0" encoding="UTC-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="contact">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="company" type="xs:string"/>
        <xs:element name="phone" type="xs:int"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



XSD Example

Add.xml

```
<?xml version="1.0" encoding="UTC-8" ?>
<contact xsi:schemalocation="Add.xsd">
    <name>Aman</name>
    <company>IBM</company>
    <phone>984155555</phone>
</contact>
```

**there will be error if we put text in between phone tag since we have set schema



XSD Attributes

All XSD attributes are declared as simple types.

Simple elements cannot have attributes. If an element has attributes, it is considered to be of a complex type. But the attribute itself is always declared as a simple type.

For eg: `<book>Science</book>` //simple element type

`<book publisher="KEC">Book</book>` // complex type

The syntax for defining an XSD attribute

`<xs:attribute name="publisher" type="xs:string"/>`

where name is the name of the attribute and xs:string specifies the data type of the attribute.

** this will only allow value of publisher be string. Like publisher="KEC" not publisher=123 or any other.

Example



Add.xsd file

```
<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:attribute name="id" type="xs:string" use="required"/>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="year" type="xs:int"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Add.xml file

```
<book xsi:schemalocation="Add.xsd">

  <book id="123">
    <title>Harry Potter and the Philosopher's Stone</title>
    <author>J.K. Rowling</author>
    <year>1997</year>
  </book>
```



XSD Attributes

Default and Fixed Values for Attributes

Attributes may have a default value OR a fixed value specified.

A default value is automatically assigned to the attribute when no other value is specified.

In the following example the default value is "EN":

```
<xs:attribute name="lang" type="xs:string" default="EN"/>
```

A fixed value is also automatically assigned to the attribute, and you cannot specify another value.

In the following example the fixed value is "EN":

```
<xs:attribute name="lang" type="xs:string" fixed="EN"/>
```

Optional and Required Attributes

Attributes are optional by default. To specify that the attribute is required, use the "use" attribute:

```
<xs:attribute name="lang" type="xs:string" use="required"/>
```



XSD Restrictions/Facets

- Restrictions are used to define acceptable values for XML elements or attributes. Restrictions on XML elements are called facets.

Restrictions on Values

- The following example defines an element called "age" with a restriction. The value of age cannot be lower than 0 or greater than 120:

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```



XSD Restrictions/Facets

Restrictions on a Set of Values

- To limit the content of an XML element to a set of acceptable values, we would use the **enumeration** constraint.
- The example below defines an element called "car" with a restriction. The only acceptable values are: Audi, Golf, BMW:

```
<xs:element name="car">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="Audi"/>
      <xs:enumeration value="Golf"/>
      <xs:enumeration value="BMW"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Restrictions on a Series of Values



To limit the content of an XML element to define a series of numbers or letters that can be used, we would use the pattern constraint.

The example below defines an element called "letter" with a restriction. The only acceptable value is ONE of the LOWERCASE letters from a to z:

```
<xs:element name="letter">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-z]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Restrictions on a Series of Values



The example below defines an element called "letter" with a restriction. The acceptable value is zero or more occurrences of lowercase letters from a to z:

```
<xs:element name="letter">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="([a-z])*"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Restrictions on a Series of Values



The only acceptable value is male OR female:

```
<xs:element name="gender">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="male|female"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

The next example defines an element called "password" with a restriction. There must be exactly eight characters in a row and those characters must be lowercase or uppercase letters from a to z, or a number from 0 to 9:

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z0-9]{8}"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```



Complex type Element

- A complex element is an XML element that contains other elements and/or attributes.
- There are four kinds of complex elements:
 1. empty elements
 2. elements that contain only other elements
 3. elements that contain only text but with attribute.
 4. elements that contain both other elements and text

Note: Each of these elements may contain attributes as well!



Complex type Element

A complex XML element, "product", which is empty:

```
<product pid="1345"/>
```

A complex XML element, "employee", which contains only other elements:

```
<employee>
```

```
  <firstname>John</firstname>
```

```
  <lastname>Smith</lastname>
```

```
</employee>
```

A complex XML element, "food", which contains only text but with attribute:

```
<food type="dessert">Ice cream</food>
```

A complex XML element, "description", which contains both elements and text:

```
<description>
```

```
  It happened on <date lang="norwegian">03.03.99</date> ....
```

```
</description>
```

Question:



- What is XML? How can we create XML file? Explain basic structure of XML with example.
- Differentiate between XML and HTML.
- Differentiate between Database and XML.
- Differentiate between DTD and XML Schema

Xpath



- XPath is a query language used for selecting nodes from an XML document. It is commonly used in conjunction with XSLT (Extensible Stylesheet Language Transformations) and XQuery to navigate and extract specific parts of an XML document.
- XPath uses a path-like syntax to locate nodes in an XML document by traversing the document's structure. It provides a way to address individual elements, attributes, or other components within the XML hierarchy.
- XPath expressions consist of a series of steps separated by slashes (/). Each step specifies a location or condition to match within the XML document. Here are some examples of XPath expressions:

Xpath



- Select all the elements with a specific tag name:
`/root/element`: Selects all "element" nodes that are children of the "root" node.
- Select elements with a specific attribute value:
`/root/element[@attribute='value']`: Selects "element" nodes that have an attribute with the value 'value'.
- Select elements based on their position:
`/root/element[1]`: Selects the first "element" node that is a child of the "root" node.
- Select elements with specific conditions:
`/root/element[@attribute='value' and @otherAttribute='otherValue']`: Selects "element" nodes that satisfy multiple conditions.

Example



- Go to <https://easycodeforall.com/generate-xpath.html> and you can generate and test your xpath.

XML File

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
  <bookstore>
    <book category="cooking">
      <title lang="en">Everyday Italian</title>
      <author>Giada De Laurentiis</author>
      <year>2005</year>
      <price>30.00</price>
    </book>

    <book category="children">
      <title lang="en">Harry Potter</title>
      <author>J K. Rowling</author>
      <year>2005</year>
      <price>29.99</price>
    </book>
```

Xpath

`/root/bookstore/book[2]/@category`

Result

`children`

XSLT (Extensible Stylesheet Language Transformations)



- XSLT (Extensible Stylesheet Language Transformations) is a language used for transforming XML documents into different formats. It is a powerful tool for converting XML data into HTML, XML, plain text, or other structured formats.
- XSLT operates on two main components: the XML input document and an XSLT stylesheet. The XSLT stylesheet contains a set of rules and instructions that define how the XML input document should be transformed. These rules are written using XSLT syntax, which is based on XML itself.



Why do we need XSLT??

- XSLT provides the ability to transform XML data from one format to another automatically. IT is the recommended style sheet language for XML.
- XSLT is far more sophisticated than CSS. With XSLT we add/remove elements and attributes to or from the output file.
- WE can also rearrange and sort elements, perform tests and make decisions about which elements to hide and display and a lot more.
- XSLT uses Xpath to find information in an XML document

Example



student.xml

student.xml

```
<?xml-stylesheet type="text/xsl" href="student.xsl"?>
<class>
  <student>
    <firstname>Graham</firstname>
    <lastname>Bell</lastname>
    <nickname>Garry</nickname>
  </student>
  <student>
    <firstname>Albert</firstname>
    <lastname>Einstein</lastname>
    <nickname>Ally</nickname>
  </student>
  <student>
    <firstname>Thomas</firstname>
    <lastname>Edison</lastname>
    <nickname>Eddy</nickname>
  </student>
</class>
```

```
<xsl:stylesheet version = "1.0"
xmlns:xsl = "http://www.w3.org/1999/XSL/Transform">

<xsl:template match = "/class">

  <html>
    <body>
      <h2>Student List</h2>

      <table border = "1">
        <tr bgcolor="lightgreen">
          <th>First Name</th>
          <th>Last Name</th>
          <th>Nick Name</th>
        </tr>

        <xsl:for-each select = "student">

          <tr>
            <td><xsl:value-of select = "firstname"/></td>
            <td><xsl:value-of select = "lastname"/></td>
            <td><xsl:value-of select = "nickname"/></td>
          </tr>

        </xsl:for-each>

      </table>

    </body>
  </html>
</xsl:template>
</xsl:stylesheet>
```

Example



Without xsl file

With xsl file

```
<!-- <?xml-stylesheet type = "text/xsl" href = "student.xsl"?> -->
<class>
  <student>
    <firstname>Graham</firstname>
    <lastname>Bell</lastname>
    <nickname>Garry</nickname>
  </student>
  <student>
    <firstname>Albert</firstname>
    <lastname>Einstein</lastname>
    <nickname>Ally</nickname>
  </student>
  <student>
    <firstname>Thomas</firstname>
    <lastname>Edison</lastname>
    <nickname>Eddy</nickname>
  </student>
</class>
```

Student List

First Name	Last Name	Nick Name
Graham	Bell	Garry
Albert	Einstein	Ally
Thomas	Edison	Eddy

Xquery



- XQuery is a query and functional programming language designed for working with XML data. It allows you to query, extract, transform, and manipulate XML documents or XML databases.
 - XQuery is *the* language for querying XML data
 - XQuery for XML is like SQL for databases
 - XQuery is built on XPath expressions
 - XQuery is supported by all major databases
 - XQuery is a W3C Recommendation
- FLWOR Expressions: FLWOR (For-Let-Where-Order by-Return) expressions are a fundamental construct in XQuery. They allow you to iterate over sequences of data, apply filters, perform calculations, and generate results.

Here is an example of what XQuery could solve:

- "Select all CD records with a price less than \$10 from the CD collection stored in cd_catalog.xml"

Example



- Consider the following XML document representing a list of employees:

```
<employees>
  <employee>
    <name>John Smith</name>
    <position>Manager</position>
    <department>Marketing</department>
    <salary>50000</salary>
  </employee>
```

```
<employee>
  <name>Jane Doe</name>
  <position>Developer</position>
  <department>IT</department>
  <salary>60000</salary>
</employee>
<employee>
  <name>Mike Johnson</name>
  <position>Analyst</position>
  <department>Finance</department>
  <salary>55000</salary>
</employee>
</employees>
```



Example

- Suppose you want to retrieve the names and salaries of employees who earn a salary greater than \$55000. You can use a FLWOR expression in XQuery to accomplish this:

for \$employee in doc("employees.xml")/employees/employee

where \$employee/salary > 55000

return concat(\$employee/name, ": \$", \$employee/salary)

For the given XML document, the result of the XQuery query would be:

Jane Doe: \$60000

Explanation



In this XQuery example:

- The `doc("employees.xml")` function is used to load the XML document from a file named "employees.xml". Adjust the filename and path as per your XML file's location.
- The for loop iterates over each employee element in the XML document.
- The where clause filters the employees based on the condition `$employee/salary > 55000`, selecting only those with a salary greater than \$55000.
- The return statement concatenates the employee's name, a colon, and their salary using the `concat()` function.
- The result of the XQuery query would be a sequence of strings displaying the names and salaries of the selected employees.

Question



- What is Xpath, XSLT and Xquery? Give proper example of each one.